OMNISTAT / COMPETITIVE INTELLIGENCE & PRODUCT STRATEGY

OmniStat 2030: Competitive Intelligence & Advanced Product Architecture

An evidence-based research paper and engineering blueprint for a next-generation restaurant intelligence platform

Prepared for the OmniStat Engineering & Product Team

12 November 2026

Contents

Product and competitor research	Technical and delivery reference
Executive abstract	10. Multi-stage engineering roadmap
1. Research design and boundaries	11. Metrics and definitions
2. OmniStat's existing foundation	12. Competitive differentiators
3. Competitive landscape	13. Product validation and market research
4. Competitor product dossiers	14. Quality engineering and verification
5. What advanced intelligence means	15. Risks and tradeoffs
6. Transferability catalog (129 features)	16. First 90-day agenda
7. Eight proposed product blueprints	17. Research gaps and future studies
8. Technical architecture	18. Final recommendations
9. Feature prioritization	Appendices A-C and bibliography (39 sources)

This is a navigable research reference; feature availability varies by vendor, plan and region. Candidate capabilities are proposals until independently validated.

Executive abstract

OmniStat can move beyond a CSV-powered sales dashboard without trying to reproduce every point-of-sale terminal, payment processor, and accounting suite in the market. A more defensible ambition is to become the **trusted restaurant decision layer**: ingest and normalize operational data from many sources; maintain consistent definitions of sales, dates, returns, inventory, and costs; expose audit-ready explanations; forecast likely outcomes; and guide the next human decision. This research analyzes publicly documented capabilities across restaurant POS platforms, back-office suites, inventory specialists, labor systems, delivery aggregators, and horizontal analytics technologies. The resulting feature inventory is broad but intentionally not exhaustive; rapidly changing enterprise products differ by region, subscription tier, integration, release date, and account configuration.

Three findings dominate. First, established restaurant platforms combine **transaction-level reporting with workflow context**—sales, orders, labor, menu, kitchen, guest, and payment events—rather than treating a chart as the end product. Toast Reporting and Square demonstrate extensive reporting breadth; Lightspeed shows product-level drill-down and menu quadrants; Restaurant365 connects financial, labor, forecasting, and inventory views. [R01–R03, R05–R12] Second, specialized providers show how **costs and operations** become actionable: invoice capture, recipe costing, theoretical-versus-actual usage, price alerts, purchasing, and labor budget controls. [R13–R21] Third, modern BI platforms increasingly depend on **governed metric definitions**, permissions, lineage, natural-language exploration, alerting, and explainable insights. AI without a trustworthy semantic layer invites incorrect financial answers. [R24–R33]

The proposed architecture therefore emphasizes a versioned metric registry, tenant-scoped ingestion and reconciliation, a provenance graph, a reporting-day policy, an evidence-backed investigation workspace, a modular analytics system, event-driven operational alerts, and—only when data sufficiency is proven—forecasting and recommendation engines. It does **not** propose inventing food margins from revenue-only CSVs, assuming order values without order IDs, or claiming customer retention without identity data. Such measures require explicit source and quality contracts.

The recommended sequence is: **(1) data trust, (2) high-utility multi-level analytics, (3) operational integrations and financial controls, (4) responsible prediction and AI, (5) automation, benchmarking, and platform extensibility**. The feature catalog, system design, acceptance criteria, test strategy, and multi-phase delivery backlog below are intended to serve as the engineering team's long-lived reference.

1. Research design, boundaries, and definitions

1.1 Research question

What capabilities do leading restaurant operations and analytics products publicly describe, which of those capabilities are technically and commercially transferable to OmniStat, and in what order should a small engineering team develop them to maximize customer usefulness while preserving data correctness, security, and operational feasibility?

1.2 Method and evidence classification

This is a **desk-research and product-architecture study**, not a hands-on competitive usability benchmark or a formal experimental paper. Evidence was gathered from official vendors' product pages, help centers, developer documentation, and from security / data-model documentation. Commercial product claims may describe aspirational capabilities, paid add-ons, limited regional availability, or beta features. No vendor account was purchased and no vendor claims were independently tested. Each documented competitor capability is followed by a source marker **[Rxx]** pointing to an official URL in the reference section.

Evidence grades used throughout: **D = directly documented** on a vendor product/help page; **P = published product claim** with uncertain commercial availability; **I = OmniStat engineering inference/recommendation**, not a competitor claim; **V = observed in the OmniStat development conversation or supplied code snapshots**, not a live audited production deployment. A feature's presence in a table does not establish that every competitor implements it or that its feature is available on every subscription tier.

Scope: restaurant business intelligence, retail-adjacent POS analytics, data ingestion, inventory/food cost, workforce, guest insights, multi-location operations, financial controls, governance, and AI. **Out of scope for the near-term product:** processing payment-card data, replacing a certified payment terminal, promising jurisdiction-specific payroll or tax

compliance, copying competitors' protected interfaces or proprietary algorithms, or claiming market leadership without customer or market evidence.

1.3 Practical limitations

The comparison is a dated public-documentation snapshot (**12 November 2026**), not an inventory of every product SKU or feature flag. Website copy can change. No private competitor roadmaps, source code, invoices, or proprietary benchmarks were reviewed. Vendor-reported outcomes, usage figures, or percentage savings are not OmniStat results. The effort estimates and objectives in this paper are design proposals, not measurements. Before sprint planning, engineering should confirm the exact GitHub `main` branch, database schema, dependency versions, and deployed runtime; the OmniStat baseline below is grounded in the user-provided development record and archived code rather than direct access to the current production deployment.

2. OmniStat today: the foundation we can extend

OmniStat's known stack includes **Python 3.13, FastAPI, PostgreSQL, HTML/CSS/JavaScript, JWT-based authentication, CSV upload ingestion, and Pytest**. Its existing sales intelligence supports 7/30/90-day windows, signed net sales and quantities, category and product drill-downs, revenue charts, product rankings, and business-specific views. A data-quality API compares daily materialized summaries with eligible line records; the team has also built a daily discrepancy investigation with paginated source records, upload batch references, and real PostgreSQL fixture verification. Those later changes may still be on a feature branch and must not be assumed merged into `main`.

Established working conventions (V): sales-line `revenue` and `quantity` are signed; refunds decrease net totals; the imported CSV's parsed `business_date` defines the current reporting day; upload processing timestamp does not replace business date; and tenant identity must come from authenticated membership, never from an arbitrary browser-provided business ID. PostgreSQL `NUMERIC` values should be compared exactly, not through floating-point tolerance. Revenue-only files cannot support true food cost, net profit, customer count, conversion rate, or average order value without new inputs.

2.1 Capability maturity assessment

Capability area	OmniStat observed state	Confidence	Next logical step
Tenant accounts and authorization	JWT login, business-scoped member/owner dependencies	V	Policy matrix, audit log, defense-in-depth row-level security
CSV upload and provenance	Parsed source data, upload IDs, processed/pending/failed states	V	Idempotency, mapping templates, duplicate detection, replay controls
Sales analytics	Net sales, net units, date windows, category/product drill-down	V	Metric registry, comparisons, custom segments, saved views
Data quality	Daily aggregate reconciliation and status panel	V	Exception queue, investigation case lifecycle, signed evidence exports
Daily investigation	Owner-only day selection, raw source evidence, pagination	V / feature branch	Reconcile distinct financial subcomponents, root-cause suggestions, review approvals
Inventory visibility	Inventory UI exists	V	Source-backed stock movements and valuation; avoid inferred stock
Multi-location management	Not verified as production-ready	Unknown	Add location hierarchy before cross-location reporting
Cost accounting	No validated invoices or per-ingredient cost inputs	Not established	Invoice and recipe data contracts
Labor analytics	No confirmed shift/payroll feed	Not established	Clock/timecard and labor cost connectors

Capability area	OmniStat observed state	Confidence	Next logical step
Guest analytics	No confirmed guest/order-identity schema	Not established	Consent-aware order and guest feeds
Forecasting and AI	No verified production ML service	Not established	Quality gates, baselines, governed metric tools

2.2 Strategic positioning

OmniStat should initially avoid competing head-on with Toast or Square as a payments/POS provider. Those businesses bundle hardware, payment processing, onboarding, merchant services, and extensive operational applications. Instead, focus on a **POS-agnostic, small-operator-friendly analytics and integrity layer**: (a) import existing sales first; (b) explain exactly how numbers were derived; (c) unite category, item, location, channel, and day drill-downs; (d) flag suspicious variance; and (e) integrate operational inputs as customers provide them. Distinguish a *verified calculation* from a *suggestion*, and a *suggestion* from an *action* requiring approval.

3. Competitive landscape and market structure

The competitive set spans five adjacent markets. Restaurant POS suites are broad platforms; back-office suites connect purchasing, food cost, accounting and labor; single-domain specialists solve particular workflows; integration hubs unify delivery/order sources; and horizontal BI/data platforms raise expectations for governance, embedded analytics, and AI. A product may be adjacent rather than a direct substitute. The comparative lesson is about **capability patterns**, not that OmniStat must copy each product's business model.

Segment	Publicly documented examples	Why OmniStat should study them
Restaurant POS and reporting	Toast, Square for Restaurants, Lightspeed, TouchBistro	Transaction models, day close, menu analysis, role reporting, reconciliations
Finance / back-office suites	Restaurant365, MarginEdge, Craftable	Actual vs forecast, invoice automation, food cost, financial workflow
Inventory and purchasing	MarketMan, WISK	Recipe costing, inventory variance, replenishment, vendor insights
Workforce and restaurant intelligence	7shifts, Tenzo	Labor forecasting, shift scorecards, automated briefings, multi-system views
Commerce and guest data	Deliverect, Olo	Channel normalization, order data, identity-aware guest analytics
Cross-industry BI and data foundations	Looker, Tableau Pulse, Metabase, Power BI, dbt, Airbyte	Semantic metrics, self-service, tenant security, data integration, explainability

4. Competitive product dossiers

4.1 Toast: POS-native operational intelligence

Documented capabilities (D): Toast Reporting provides in-depth, near-real-time sales and operational views across channels and mobile access. Toast IQ connects conversational AI to sales, labor, menu, guest and workflow data, with both descriptive answers and some permitted operational actions. Restaurant Benchmarking compares item-level sales, volume, pricing, and hourly behavior with defined peer sets, subject to data sufficiency. xtraCHEF connects invoice line items, recipe costing, inventory, and accounting sync. [R01, R02, R03, R04, R34]

Transferable to OmniStat (I): channel-aware net-sales metric definitions; automatic shift briefing with source-backed explanations; category/item trends; benchmark *methodology* with privacy thresholds; automatic variance alerts; invoice-to-recipe cost lineage. **Not transferable without prerequisites:** peer-market benchmarking absent sufficient opted-in businesses; transactional actions in a third-party POS absent permissioned integrations; instant data without continuous event ingestion. **Competitive risk:** a unified POS vendor can correlate orders and payments more readily than a CSV-only analytics provider.

4.2 Square for Restaurants: comprehensive reporting and reconciliation

Documented capabilities (D): Square describes item/category/time/location sales, deposits/refunds/fees reconciliation, deferred-sales tracking, custom work weeks, live service-section reports, catalog reporting across channels, COGS/vendor

reporting, labor-to-sales monitoring, customer segments, marketing, and a **Square AI beta** feature. Some items are tier-dependent. [R05]

Transferable (I): robust daily reconciliation that separates **net sales** from **settlement deposits**, business-day calendars, group-by service period, and clearly labeled reports that warn when order or fee data is absent. Square's explicit distinction between when an order is placed and fulfilled is a reason to model order, fulfillment, settlement, and accounting dates separately.

4.3 Lightspeed Restaurant: menu intelligence and item scorecards

Documented capabilities (D): Lightspeed Advanced Insights includes segmented sales, server, popular-time, and menu reporting. Menu reports offer Most Popular, Repeat Customer Favorites, Magic Menu Quadrant, and a product item scorecard with sales history. Availability is documented as regional/product-specific. [R06, R07]

Transferable (I): scorecard-style drill-down, menu quadrant, popularity vs margin once costs exist, shift comparisons, and product history. A **revenue-only quadrant must not be labeled profit or margin**. OmniStat can offer volume versus net sales now and upgrade to unit contribution margin when reliable ingredient inputs exist.

4.4 TouchBistro: accessible operational reports

Documented capabilities (D/P): TouchBistro markets more than 50 near-real-time reporting views, staff reporting, spending pattern analysis, guest experience insights, marketing tie-ins, and off-premises access. This is a vendor statement, not an independently verified report count. [R08]

Transferable (I): compact role-specific reports, owner/mobile digest, staff dashboards only when shift/sales-person keys exist, and simple navigation rather than an unbounded list of charts.

4.5 Restaurant365: unified finance, labor, and forecasting

Documented capabilities (D/P): Restaurant365 combines restaurant accounting, AP, inventory, operations, labor, reporting, and AI dashboards. Its documentation describes sales/labor actual-versus-forecast dashboards, location-level forecast variance analysis, and natural-language dashboard creation; published AI capabilities and rollout statuses vary by module and time. [R09, R10, R11, R12, R35]

Transferable (I): a canonical daily profit-control view with drill-down from overview to location, category, product and evidence; forecast versus actual with reason codes; scheduled labor versus sales; month-end reconciliation checklists; versioned report snapshots. **Prerequisite:** purchase/invoice, payroll, and labor inputs must exist before accounting-style metrics are credible.

4.6 MarginEdge: automated cost control and explainable margins

Documented capabilities (D): MarginEdge supports invoice processing from photo/upload/email/EDI, vendor item mapping, plate/recipe costing, price alerts, purchase visibility, daily controllable P&L, menu analysis, and theoretical versus actual inventory usage. Its site also describes an AI assistant that helps explain price and margin movements. [R13, R14, R15]

Transferable (I): invoice line-item provenance and review; vendor price drift; recipe cost history; exception explanations that show exactly which ingredients, prices, and quantities changed. **Prerequisite:** never infer food cost solely from menu-item sales and inventory labels.

4.7 MarketMan and WISK: inventory becomes operational intelligence

Documented capabilities (D/P): MarketMan lists recipe costing, AVT reporting, waste management, invoice scanning, vendor ordering, purchasing, multi-unit transfers, and AI-supported ordering in tiered plans. WISK describes mobile counts, recipe costing, cost/stock alerts, multiple outlets, and outlet variance reporting. Plan features and usage caps vary. [R16, R17, R20]

Transferable (I): ingredient catalog, unit conversions, inventory movement ledger, count variance, purchase-order suggestions, negative-stock flags, and sales-driven theoretical consumption. **Prerequisite:** ingredient-level recipes and stock transactions, including waste, transfers and adjustments, must be captured.

4.8 Craftable: finance operations and procurement controls

Documented capabilities (D): Craftable markets purchasing/vendor centralization, price/off-contract flags, invoice intake, recipe-based depletion, inventory auditing, and analytics spanning sales, labor and purchasing. Its support center lists hourly sales, prime-cost, forecast, and profit-by-server reports. [R21, R37]

Transferable (I): vendor spend dashboards, purchase-approval workflows, three-way comparisons (purchase order, receipt, invoice) and margin bridge analysis. **Prerequisite:** supplier IDs, invoice lines, received quantities and approval roles.

4.9 7shifts: workforce-aware performance

Documented capabilities (D): 7shifts details schedules informed by sales/labor budgets, time clocks, overtime and break monitoring, tip pooling, payroll, team communication, manager logbooks, and attendance/team performance reports. [R18]

Transferable (I): forecast sales to suggested labor *hours*, overtime risk alerts, hours-versus-sales analysis, shift handover summaries and timecard anomaly queues. **Prerequisite:** clock records and location/role/pay-rate data with careful personal-information access controls.

4.10 Tenzo: cross-system insights and proactive action

Documented capabilities (D/P): Tenzo aggregates POS, workforce, inventory, reviews, reservations and other inputs; offers drag-and-drop performance exploration, AI-assisted questions, demand forecasts incorporating holidays/weather/events, automatically delivered reports, and real-time scorecards and alerts. [R19]

Transferable (I): 8 a.m. operator briefing, variance-to-cause investigation, forecast confidence bands, site comparisons, prioritized action queue, and end-to-end links from anomaly to the contributing line items. **Prerequisite:** connector coverage, time alignment, source trust and alert deduplication.

4.11 Deliverect and Olo: channels and guest identity

Documented capabilities (D): Deliverect's analytics includes order/revenue metrics and cross-filters for locations, channels, and dates; its developer platform exposes order, menu and reporting integrations. Olo's guest platform unifies POS, online orders, loyalty, reservations, payments, and other touchpoints for acquisition, retention, frequency, and guest lifetime value analysis. [R22, R23]

Transferable (I): normalized channel dimension, delivery commission comparison, unsuccessful order tracking, repeat-guest cohorts, opt-in marketing analytics and channel attribution. **Prerequisite:** order IDs, channel events, fees and lawful identity/consent collection.

4.12 Looker, Tableau Pulse and Metabase: governed self-service analytics

Documented capabilities (D): Looker emphasizes a semantic modeling layer, embedded reports and governed conversational analytics. Tableau Pulse offers personalized metric feeds, trend and driver explanations, natural-language questions and delivery through everyday workflows. Metabase supports embedded interactive dashboards, filters, drill-through, alerts, SSO and tenant-oriented permissions with distinct commercial entitlements. [R24, R25, R26]

Transferable (I): shared metric catalog, saved reports, governed natural-language query generation, explainable alerts, subscriptions, embedded visual building blocks, and strong tenant isolation. The critical transferable concept is **consistent metrics**, not replacing OmniStat with a generic BI suite overnight.

4.13 Power BI, dbt and Airbyte: reliability and extensibility patterns

Documented capabilities (D): Power BI supports row-level security and incremental data refresh. dbt provides centralized metrics, source freshness checks, and reusable database assertions. Airbyte provides a large connector ecosystem and change-data-capture pathways. [R27–R32]

Transferable (I): incremental sync, source-to-metric lineage, metric version control, automated SQL quality assertions, safe customer row isolation, data freshness status, and connector contracts. Avoid premature warehouse complexity while a well-indexed PostgreSQL deployment remains sufficient for the workload.

5. Research synthesis: what 'advanced' means in practice

An advanced product is not a dashboard with dozens of chart libraries. It is a system that has **(a) broad enough data, (b) trusted computation, (c) useful explanation, (d) a timely intervention, and (e) measurable improvement**. The same visual pattern can be valuable or misleading depending on the source data and accounting semantics. Revenue charts built from synthetic or incomplete extracts may look impressive but cannot substitute for a reconciliation with payment fees, tax, discounts, refunds, and settlements.

An evidence-backed OmniStat story should connect six steps: **connect → validate → define → analyze → explain → act**. For example, when Tuesday net sales decline, the system should identify which locations/categories/products contributed, link them to the exact source sales and import batches, check missing-data warnings, compare historical or forecast baselines, and propose a review action. If it cannot prove the cause, it must say *unexplained* and show the evidence, not fabricate a cause.

6. Comprehensive transferability catalog

The following catalog groups **proposed OmniStat capabilities**, including existing or partially built features. It does not assert that every referenced vendor has exactly the listed feature. The *inspiration* field links to supporting official descriptions of the underlying capability pattern, or I only when it is a design proposal without a specific vendor claim. *Priority* means sequencing recommendation: **P0 foundational; P1 near-term value; P2 selective expansion; P3 long-horizon/conditional**. A Conditional status explicitly means missing source inputs or dependencies must be acquired before the metric can be shown reliably.

6.1 Data connection, ingestion and source provenance

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Connector wizard with CSV profile detection	Partial	**P0**	CSV mapping samples	[R22, R32]
Column mapping templates by POS export	Proposed	**P0**	Confirmed export samples	[R05, R22]
Scheduled file import from approved source	Proposed	**P1**	Service account / signed upload	[R32]
Incremental POS API synchronization	Proposed	**P2**	Vendor API access	[R22, R32]
Webhook ingestion with signature verification	Proposed	**P2**	Vendor webhooks	[R22, R33]
Idempotent import and row-level duplicate prevention	Partial	**P0**	Stable import fingerprint	[R29, R30]
Raw source quarantine and parsing error UI	Proposed	**P0**	Raw file retention policy	[R30]
Source lineage per sale, batch and field	Partial	**P0**	Persistent upload and row IDs	[R29]
Schema drift and field mapping version alerts	Proposed	**P1**	Schema contracts	[R30, R31]
Backfill / replay with dry run	Proposed	**P1**	Immutable raw snapshots	[R32]
Connector health and freshness dashboard	Proposed	**P1**	Scheduled/stream input	[R31, R32]
Business-day/calendar rules per location	Partial	**P0**	Timezone and close-hour policy	[R05]
Currency normalization and exchange provenance	Conditional	**P2**	Multi-currency sources	I only
Data ownership and deletion/export controls	Proposed	**P1**	Privacy policy	[R33]

Engineering interpretation. This domain contains 14 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.2 Revenue integrity, reconciliation and financial investigation

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Source-versus-summary daily reconciliation	Existing / extend	**P0**	Signed sale-line revenue	[R05, R09]
Evidence-backed daily discrepancy drill-down	Feature branch / verify	**P0**	Upload IDs and line rows	[R05, R09]
Exception queue with owner and status	Proposed	**P0**	Persistent discrepancy cases	[R09, R24]
Categorized mismatch reasons with confidence	Proposed	**P1**	Deterministic reconciliation rules	[R09]
Gross, discount, tax, tip, fee and net bridge	Conditional	**P0**	Separate POS fields	[R05]
Payment settlement / bank deposit matching	Conditional	**P2**	Payment and payout feeds	[R05]
Refund / void / comp anomaly analysis	Conditional	**P1**	Event types, staff keys	[R02, R05]
Missing business-day and late-arrival alerts	Proposed	**P0**	Data completeness rules	[R31]
Line-level source and import trace	Partial	**P0**	Provenance FK integrity	[R29]
Approval evidence pack and signed export	Proposed	**P2**	Audit log, export jobs	[R09]
Financial close readiness score	Proposed	**P2**	Resolved checks by day	[R09]
Revenue restatement and metric version history	Proposed	**P1**	Metric versioning	[R29]
Read-only reviewer role	Proposed	**P1**	Role matrix	[R27, R33]
Cross-location exception rollup	Conditional	**P2**	Location dimension	[R09]

Engineering interpretation. This domain contains 14 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.3 Sales intelligence, commercial analytics and menu design

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Net sales, net units and growth windows	Existing	**P0**	Signed sales lines	[R01, R05]
Category and product drill-down pages	Existing	**P0**	Canonical item/category	[R06, R07]
Weekpart and hourly demand heatmaps	Conditional	**P1**	Event timestamps	[R06]
Location / channel / service-period pivots	Conditional	**P1**	Location and channel fields	[R22]
Menu engineering matrix: volume x margin	Conditional	**P1**	Reliable recipe cost	[R07, R13]
Basket affinity / items bought together	Conditional	**P2**	Order-line identity	[R02]
Item modifier / option performance	Conditional	**P2**	Modifier event data	[R02]
Promotion and discount lift analysis	Conditional	**P2**	Promotion attribution data	[R05]
Price elasticity experiments	Conditional	**P3**	Historical prices and exposure	I only
Sales contribution and variance waterfall	Proposed	**P1**	Comparable grain and periods	[R25]
Repeat-customer menu favorites	Conditional	**P2**	Consent-aware guest IDs	[R07, R23]
Saved custom filters / segments	Proposed	**P1**	Semantic metric registry	[R24, R26]
Scheduled owner digest and PDF export	Proposed	**P1**	Alert and export service	[R01, R25]

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Same-store comparisons / cohort normalization	Conditional	P2	Location lifecycle metadata	[R09]
Margin sensitivity / scenario explorer	Conditional	P2	Price and cost inputs	[R10, R13]

Engineering interpretation. This domain contains 15 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.4 Inventory, procurement, recipes and cost intelligence

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Canonical ingredient SKU catalog	Proposed	P1	Unit and vendor IDs	[R16, R20]
Recipe bill of materials and yields	Proposed	P1	Ingredient quantities	[R13, R16]
Live recipe / plate cost history	Conditional	P1	Price-per-unit invoice history	[R13, R15]
POS-driven theoretical ingredient depletion	Conditional	P2	Recipes + item-level orders	[R13, R16]
Cycle counts / physical inventory	Proposed	P1	Inventory ledger	[R16, R20]
Actual versus theoretical food usage	Conditional	P2	Counts, recipes, waste, purchases	[R13, R16]
Waste and spoilage event tracking	Proposed	P1	Waste categories / quantities	[R16]
Cross-location stock transfer ledger	Conditional	P2	Locations + stock balance	[R16, R20]
Vendor invoice capture and line extraction	Proposed	P2	OCR approval & correction UI	[R14, R21]
Invoice price variance alerts	Conditional	P1	Reliable supplier price feed	[R15, R21]
PO creation and approvals	Proposed	P2	Vendor catalog, permissions	[R16, R21]
Reorder recommendations with confidence	Conditional	P2	Demand, lead time, safety stock	[R16]
Food cost, COGS and margin bridge	Conditional	P2	Cost allocation policy	[R09, R13]
Allergen and recipe document control	Conditional	P3	Verified recipe data	[R13]

Engineering interpretation. This domain contains 14 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.5 Labor, workforce and scheduling intelligence

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Labor cost / net sales ratio	Conditional	P1	Actual payroll cost and net sales	[R18, R12]
Scheduled versus actual labor hours	Conditional	P1	Timecards + schedules	[R18]
Overtime and break-risk indicators	Conditional	P2	Jurisdictional work rules	[R18]
Demand-aware staff suggestions	Conditional	P2	Forecasts + staff availability	[R12, R18]
Sales per labor hour / employee	Conditional	P1	Clock events and labor scopes	[R18]
Manager shift handover logbook	Proposed	P1	Role/shift entities	[R18]
Attendance anomaly and clock corrections	Conditional	P2	Clock event audit trail	[R18]

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Tip-pooling reconciliation	Conditional	P3	Tip rules and payroll integration	[R18]
Shift staffing vs hourly traffic	Conditional	P2	Hourly time and demand	[R12]
Workforce privacy/access separation	Proposed	P0	Sensitive employee data policy	[R33]

Engineering interpretation. This domain contains 10 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.6 Guest, loyalty and marketing intelligence

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Consent-aware first-party guest profiles	Conditional	P2	Lawful guest identifiers	[R23]
Visit frequency and recency cohorts	Conditional	P2	Order-to-guest linking	[R23]
RFM segmentation	Conditional	P2	Guest transaction history	[R23]
Guest lifetime value estimate	Conditional	P3	Identity, horizon, margin assumptions	[R23]
Churn risk / reactivation cohorts	Conditional	P3	Consented longitudinal data	[R23]
Campaign performance and attribution	Conditional	P2	Marketing message + conversion IDs	[R05, R23]
Review sentiment and topic trends	Conditional	P2	Approved review source	[R19]
Reservation and table utilization	Conditional	P2	Reservation/seat data	[R23]
Guest access/deletion requests	Proposed	P1	Privacy data inventory	[R23, R33]

Engineering interpretation. This domain contains 9 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.7 Operational efficiency, delivery and service quality

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Delivery channel P&L decomposition	Conditional	P2	Marketplace fees and orders	[R22]
Cancelled, failed or refunded order view	Conditional	P1	Order state transitions	[R22]
Kitchen ticket cycle-time distribution	Conditional	P2	KDS event timestamps	[R05]
Peak staffing bottleneck maps	Conditional	P2	Kitchen/service time events	[R05, R19]
Menu availability / 86 event tracking	Conditional	P2	POS stock event integration	[R02]
Cross-channel sales comparison	Conditional	P1	Channel normalization	[R22]
Prep forecast and waste safeguards	Conditional	P2	Sales + recipe + prep history	[R12, R13]
Ops incident timeline	Proposed	P2	Event/audit ingestion	[R19]
Shift scorecards and runbooks	Proposed	P1	Target thresholds	[R18, R19]
Automated close-of-day checklist	Proposed	P1	Exception and task model	[R05]

Engineering interpretation. This domain contains 10 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.8 AI, forecasting and decision support

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Natural-language query over governed metrics	Proposed	**P2**	Metric registry + safe query compiler	[R02, R24, R29]
Explain KPI change by contributing dimensions	Proposed	**P1**	Variance decomposition	[R25]
Constrained daily briefing with source citations	Proposed	**P1**	Trusted metrics and alert feed	[R02, R19]
Anomaly alerting with predictable false-positive rate	Proposed	**P1**	Seasonal baseline & QA	[R25]
Short-horizon sales forecasting	Conditional	**P2**	Sufficient dated history	[R12, R19]
Forecast uncertainty bands and backtests	Proposed	**P2**	Forecasting evaluation pipeline	I only
Weather, events and holiday covariates	Conditional	**P3**	Licensed external signals	[R19]
What-if pricing/staffing scenarios	Conditional	**P2**	Causal assumptions and inputs	[R10]
Root-cause hypotheses, never fabricated causes	Proposed	**P1**	Source lineage + reason rules	[R25]
Evidence-linked recommendations with owner review	Proposed	**P2**	Approval workflow	[R02]
Forecast drift, leakage and quality monitoring	Proposed	**P2**	Backtest + monitoring datasets	I only
Prompt/SQL safety and red-team evaluation	Proposed	**P0**	Safe tool permissions	[R24, R33]
Alert fatigue and escalation policy	Proposed	**P1**	Notification preferences	[R19]

Engineering interpretation. This domain contains 13 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.9 Collaboration, reporting, alerts and workflow

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Saved investigations with comments	Proposed	**P1**	Case IDs and audit log	[R26]
Assign anomaly to owner or reviewer	Proposed	**P1**	User/role model	[R18]
Scheduled morning digest and weekly pack	Proposed	**P1**	Background jobs + templates	[R19, R25]
Threshold and relative-change alerts	Proposed	**P1**	Rule engine	[R19]
Low-noise snooze/acknowledge workflow	Proposed	**P1**	Notification state	[R19]
Dashboard annotations and KPI explanations	Proposed	**P1**	Metadata store	[R25]
Role-based share / export limits	Proposed	**P1**	RBAC and data labels	[R26, R27]
Versioned weekly report snapshots	Proposed	**P2**	Metric versions and export job	[R24]
Push / email / mobile delivery	Conditional	**P2**	Verified addresses and consent	[R25]

Engineering interpretation. This domain contains 9 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.10 Platform scale, developer experience and governance

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Location hierarchy and multi-unit rollups	Conditional	**P1**	Locations under businesses	[R09, R16]
Tenant-level row-level security in PostgreSQL	Proposed	**P0**	Session tenant propagation	[R27, R33]
Fine-grained roles and reviewer permissions	Proposed	**P0**	Policy matrix	[R26, R33]
SSO and enterprise account provisioning	Conditional	**P3**	Enterprise customers	[R26]
Read-only customer API and webhooks	Proposed	**P2**	OAuth/scoped tokens	[R22]
Change-data/event outbox and retry queue	Proposed	**P1**	Transactional event design	[R32]
Metric registry with semantic versioning	Proposed	**P0**	Metric contracts	[R24, R29]
Immutable audit trail for financial actions	Proposed	**P0**	Append-only events	[R33]
Data retention, deletion, and secure exports	Proposed	**P1**	Customer policy & key management	[R33]
Query cost caps, indexes and read replicas	Conditional	**P2**	Observed performance needs	[R28]
Feature flags, migrations and tenant canaries	Proposed	**P1**	CI/CD and observability	I only
Mobile-ready responsive dashboards	Partial	**P1**	Accessibility and performance	[R08, R25]
Accessibility and keyboard navigation	Proposed	**P0**	WCAG review	I only

Engineering interpretation. This domain contains 13 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.11 Commercial onboarding, verticalization and packaging

Candidate feature	OmniStat status	Priority	Data / engineering prerequisite	Inspiration
Five-minute CSV quick start	Proposed	**P0**	High-quality example mappings	I only
Customer-specific data confidence score	Proposed	**P1**	Quality and freshness rules	[R30, R31]
Restaurant/retail vertical templates	Proposed	**P1**	Industry schemas	[R01, R05]
Free diagnostic report / upgrade path	Proposed	**P2**	Unit economics and pricing research	I only
Help center and in-product explanations	Proposed	**P1**	Stable metric dictionary	[R08]
Integration catalog / partner program	Conditional	**P3**	API reliability and demand	[R22, R32]
Privacy-first market benchmark cohorts	Conditional	**P3**	Sufficient opted-in sample	[R03]
Operational onboarding checklist	Proposed	**P1**	Tenant provisioning	I only

Engineering interpretation. This domain contains 8 distinct candidate features. Implement the P0 controls before expanding presentation. Conditional features should remain hidden, explicitly unavailable, or marked as experimental until their prerequisite fields, schema migrations, validation datasets, and tenant permissions exist.

6.12 Summary of breadth and priority

Priority	Count	Interpretation
P0	22	Trust and safe baseline
P1	51	Near-term usable workflows
P2	47	Data-dependent expansion

Priority	Count	Interpretation
P3	9	Long-horizon experimentation

Total cataloged capabilities: 129. These are a research inventory, not a commitment to build them all. Estimate each feature against available sources, operator demand, data rights, maintenance overhead and security risk. Prioritizing an evidence-backed 20-feature product will outperform shipping 100 unverified UI controls.

7. Eight reference product blueprints

7.1 TrustGraph: the data integrity and financial investigation system

User problem. A restaurant owner sees $1,800 in the dashboard but $1,940 in an uploaded POS report. They cannot tell whether a return was booked differently, a file was double-imported, a date boundary shifted, or a daily aggregate is stale. A top-level red warning is not an investigation workflow. OmniStat should organize each exception as a traceable case with known evidence, unknowns, and a safe resolution path.

Proposed experience. From any KPI tile, the operator selects a day and reaches a four-part investigation: *reported number*, *recomputed number*, *signed delta*, and *confidence/evidence*. The next panel gives reason categories (missing summary, quantity-only mismatch, suspected duplicate, late file, unrecognized category, refund timing), each classified as confirmed, hypothesis, or not enough data. The evidence table shows signed line amounts, original upload, source row, location, day, parsed fields, and ingestion timestamp; filters expose suspicious subsets. A reviewer can assign a case, annotate the investigation, acknowledge a false positive, and export a snapshot—but an investigation never silently rewrites raw sales.

Core data design. Use append-only ingestion_batches, source_rows, reconciliation_runs, reconciliation_cases, case_evidence, case_events, and metric_versions. A source row is uniquely identified by (business_id, source_system, source_record_key, source_record_version) or by a documented import-line fingerprint when the vendor lacks stable IDs. A reconciliation run records the *metric definition version*, *reporting-day rule version*, *input snapshot / watermark*, *cutoff*, and *SQL result hashes*. Cases represent unresolved differences; individual runs provide point-in-time reproducibility. Key question: is the discrepancy an accounting difference or a data-quality difference? Keep those types separate.

API sketches (proposal): GET /v1/reconciliation/days?from=&to=, GET /v1/reconciliation/days/{day}, GET /v1/reconciliation/cases/{id}, GET /v1/reconciliation/cases/{id}/evidence?cursor=, and POST /v1/reconciliation/cases/{id}/annotations. The last endpoint edits **case metadata only**; source financial evidence remains read-only. Each response includes business_id from server-side membership context, metric versions, status reason, exact display amounts, and pagination cursors. The existing /api/business/data-quality/day is a compatible starting point; a wholesale API rewrite is unnecessary.

Acceptance criteria. Exact Decimal / NUMERIC agreement on a known seed; negative refunds reduce net sales; an absent summary remains null, not $0; repeated imports do not inflate revenue; the source and aggregate use one repeatable snapshot; the API never trusts a request's business ID; every row traces to an allowed tenant's import; signed evidence is exportable without PII; and the user sees a precise empty state. A full flow must pass in a real seeded PostgreSQL database, not only a mock.

Differentiation. Stronger auditability than a generic revenue chart: OmniStat should make the numeric claim testable, make the evidence accessible, and explicitly acknowledge when no known root cause has been proven.

7.2 InsightAtlas: configurable multi-level analytics

User problem. Owners need simple answers first, then different degrees of depth without leaving their business context. A top-ten item list hides location, time, channel, and category differences. A huge single page creates cognitive overload.

Proposed hierarchy. Business → Location → Channel → Department → Category → Product → Modifier → Source records. Every screen shares the same date and metric contract and uses a visible breadcrumb. If a user clicks a category segment, every linked chart filters to the same authorized subset. Saved views capture the filter

state, tenant scope, comparison period, and metric version rather than hand-writing distinct SQL into each JavaScript widget.

High-value chart families. Trend with a comparable previous period; day/hour heatmap where event timestamps exist; net-sales waterfall decomposing category contribution; ranked product distributions with cumulative share; cohort of repeat customers if guest IDs exist; dispersion of performance by site; site/shift scatterplots; uncertainty-aware forecasts; menu positioning only after unit cost is reliable; and a cost bridge linking invoice price movements to recipe margin. Always provide a table view and a CSV export with units, dates and data freshness. A chart library is an implementation detail, not the product outcome.

Engineering model. Define one metric-query contract with `metric`, `grain`, `group_by`, `filters`, `comparison`, `time_zone`, `date_policy_version`, `limit`, and `sort`. Compile a validated allowlist into parameterized SQL or queries against approved materialized rollups. Do not offer raw SQL to untrusted users. Cache keys must include tenant, metric version, date policy and permissions; otherwise users may see inconsistent or cross-tenant results. Provide `GET /v1/metrics/catalog` and `POST /v1/analytics/query` as future interfaces, not current endpoints.

Acceptance criteria. On a frozen dataset, business net sales exactly equals the sum of disjoint location/department/category slices when classification is complete and each record maps to one slice. For missing categories, explicitly expose `Uncategorized`; do not discard them to make the product mix total look correct. Negative-only category shares and zero-denominator growth remain labeled, not misleading percentages.

7.3 MarginLens: recipe, inventory and procurement economics

User problem. A business can sell more and earn less if ingredient prices increase. A sales-only dashboard cannot reveal this. MarginEdge, MarketMan, WISK, and Craftable demonstrate how much value comes from treating invoices, recipes and stock movements as first-class data. [R13–R17, R20–R21]

Phased plan. Start with ingredient master data, units of measure, vendor mapping, invoice line capture, recipe ingredient quantities, yield and waste rules. Establish versioned recipe costing where a dish's estimated material cost at a timestamp is the sum of ingredient quantity multiplied by the corresponding effective unit cost, adjusted for yield and known prep loss. A separate **actual COGS** measure requires an agreed accounting method and inventory movement records. Do not label estimated recipe ingredient cost as actual COGS. The cost bridge breaks observed changes into ingredient price, portion/yield, mix, waste and purchasing components only when evidence exists.

Useful actions. Highlight a vendor whose unit price increased 12% relative to its previous invoice, identify impacted dishes, show a price timeline, and propose a review of recipe, supplier or pricing—not an automatic price change. Reorder suggestions combine expected demand, lead time and safety stock with a manual approval path. If the cost feed is stale, all profit screens show a freshness warning.

Acceptance criteria. Unit conversions are tested (g/kg, ml/l, case/each), fractional yields are explicit, invoices cannot silently replace menu revenue, stock balances are reconstructible from movements, and every cost estimate records a source/version. Users can explain the difference between theoretical usage, physical counts and actual purchase consumption.

7.4 LaborPulse: demand-aligned staffing intelligence

User problem. Restaurants often schedule based on intuition despite predictable sales patterns. 7shifts and Restaurant365 describe forecast-informed labor budgets, schedules, timecards, compliance monitoring and hourly actual-versus-forecast views. [R11, R12, R18]

Data prerequisites. Build a time dimension and location-specific business-day cutoff; ingest or model clock-in/out, breaks, shifts, pay rates, expected roles, coverage rules, scheduled hours, and legally applicable overtime parameters. Maintain separate views of planned wage cost, actual wage cost, and estimated wage cost; one must not masquerade as another. Access to individual employee performance requires granular permission and privacy review.

Planning workflow. Predict location/hour sales and uncertainty, convert to suggested minimum role coverage using operator-owned staffing ratios and floor constraints, compare labor budget to schedule, and flag risk. An operator may edit and approve a suggestion. Scheduling should remain a recommendation surface unless OmniStat has robust bidirectional integration with the scheduling system.

Evaluation. Compare predicted versus actual demand over time, labor-to-sales performance under comparable operating conditions, schedule change acceptance, overtime incidents, and employee impact. A model that reduces labor expense by understaffing the floor should **not** count as success; service, safety and staff satisfaction are guardrail outcomes.

7.5 ForecastStudio: demand forecasting with calibrated uncertainty

User problem. A flat growth percentage is not a plan. Operators want to know likely sales next shift, expected busy windows, and what to prep—while understanding forecast uncertainty. Tenzo and Restaurant365 report demand/forecast functionality, but the inputs vary across modules. [R12, R19, R35]

Minimum responsible approach. Establish a baseline that predicts tomorrow from comparable weekdays and recent seasonality. Validate with rolling-origin backtests across multiple stores and calendar periods. Prefer WAPE, MAE and MASE over unqualified MAPE on low/zero-sales data. Publish a signed forecast error table, 50/80/95% interval coverage, and fallbacks for sparse products. Forecast by location/day first; only forecast hour/product when enough history exists. Start with classical/statistical models before adding heavier ML; more parameters are not proof of better forecasts.

Feature design. Calendar, promotions, holidays, opening hours, outages and historical sales are first-party input candidates. Weather and local-event data become optional covariates only with lawful usage rights and verified incremental value. Store the model version, training cutoff, freshness status, evaluation window, feature list, prediction/interval and subsequent actual. Future recommendations about procurement or staffing must cite the exact forecast run and its confidence.

Acceptance criteria. Demonstrate out-of-sample improvement over a naive comparable-weekday baseline with confidence intervals, no future leakage, correct behavior under partial-day data, an explicit missing-data fallback, and no automatic operational actions without approval.

7.6 AskOmni: grounded conversational analytics and explanations

User problem. A business owner asks “Why did beverage revenue fall last Tuesday?” and expects a trusted, verifiable answer, not an eloquent hallucination. Toast IQ, Looker, Restaurant365 AI dashboards and Tableau Pulse demonstrate growing demand for conversational and automatically explained analytics. [R02, R10, R24, R25]

System design. The natural-language interface parses intent into an approved metric, group-by fields, date range and filters. The backend validates a tenant-specific query plan against an allowlist, executes read-only SQL with limits, and returns a structured evidence object: rows, source IDs, metric definitions, period comparison, warnings and uncertainty. A language model can explain the returned evidence but must **never** generate arbitrary production SQL executed without checks. For inferred causes, require an explicit `hypothesis` label and supporting-versus-conflicting evidence. Data confidence and freshness travel with every answer.

Example safe answer. “Beverage net revenue was $310 lower than the prior Tuesday. I can attribute $190 to lower units of iced coffee and $120 to other beverage products; the provided data does not establish why demand changed. Sources: 86 sales lines from two approved import batches; data freshness 08:30 local.” This is an illustrative answer format, not a measured OmniStat fact.

Evaluation. Maintain a benchmark set of operator questions with exact SQL expected answers, insufficient-data cases, cross-tenant adversarial prompts, prompt-injection attempts in CSV fields, and sensitive-data requests. Score metric correctness, provenance coverage, refusal quality, P95 latency, hallucination rate and tenant leakage. Ship read-only mode before any action-taking agent.

7.7 ActionHub: insight-to-workflow without unsafe autonomy

User problem. Alerts become noise unless operators can assign, acknowledge and resolve them. A useful workflow might connect a missing summary to an assigned reviewer, a vendor price rise to a purchasing approval, or a forecasted rush to a staffing review. [R18, R19, R25]

Proposed structure. `signals` describe observed facts with an evidence URL; `cases` track investigation ownership and state; `recommendations` propose a response with expected outcome and uncertainty; `approvals` record explicit human decisions; `outbox_events` deliver authorized integrations. Every automated notification has severity, expected recipient, snooze, dedupe key, repeat policy, and retention. Separate the alert engine from the financial ledger. No recommendation is automatically applied to a POS, payroll system or purchasing account without a permitted API and owner-approved action scope.

Measurement. Alert open rate alone is not an outcome. Measure time-to-detect and time-to-resolve known data issues, percentage of actionable alerts, false-positive rate, acknowledgment latency and follow-through, ideally alongside business outcomes in controlled pilots.

7.8 OmniStat Platform: multi-location, integrations and governance

User problem. Operators scale from one business to many locations; the same category name or product may differ by site, POS, recipe or price. A single `business_id` column is necessary but insufficient for robust enterprise workflows. Restaurant365, MarketMan, Tenzo and modern BI tools show the importance of location hierarchy, access policy and reusable metrics. [R09, R16, R19, R24, R27]

Design. Introduce `organization → business → location` relationships with explicitly scoped memberships and roles. Support effective-dated menu/product mappings, standardized and local categories, source-system identities, permission templates, event audits and versioned metric contracts. Public APIs need stable versioning, per-tenant rate limits, secret rotation, durable webhooks and scoped service accounts. Start in the existing FastAPI/PostgreSQL system; add a job queue/cache/read replica only when measurements show a bottleneck.

Security validation. For every endpoint and export, test user A's valid object ID against tenant B's record; reject at the database query boundary. Enforce `business_id` and location predicates in SQL, plus optional PostgreSQL row-level security as defense in depth. Document lifecycle of personal information; collect the least data necessary and avoid copying identifiable guest or employee fields into diagnostic logs. OWASP highlights broken object-level authorization as a major API failure mode. [R27, R33]

8. Technical architecture blueprint

8.1 Reference architecture: modular monolith first

The most appropriate near-term design is a **well-structured modular monolith**: one FastAPI deployable with isolated submodules for ingestion, identity, sales metrics, reconciliation, inventory, jobs, forecasts, and exports. PostgreSQL remains the system of record. A light job runner can process asynchronous imports and scheduled digests; object storage is useful for raw CSVs and evidence files if secure lifecycle rules are implemented. An event outbox prevents the database commit from becoming inconsistent with a notification. Avoid introducing Kafka, microservices, a feature store, and a data warehouse simultaneously before customer scale warrants them.

Proposed logical path:

POS CSV / permitted API / invoices / schedules → Source adapter and immutable batch manifest → Validated, normalized tenant-scoped fact tables → Quality tests and versioned semantic metrics → Aggregates and reconciliation → FastAPI REST / approved metric query API → Business / category / product / daily evidence dashboards → Forecast / explanation / recommendations → Human approval / export / notification.

There are three orthogonal controls spanning this path: **identity and tenant authorization**, **lineage and auditability**, and **freshness / quality observability**. AI consumes the governed metric service and never bypasses it to read arbitrary raw tables. Streaming/near-real-time is a later enhancement; initially a transparent "last refreshed" timestamp is more honest and cheaper to deliver.

8.2 Proposed core data contracts

Entity / fact	Keys and critical columns	Required invariants
businesses, locations	ID, parent, timezone, business-day start hour	Membership and location scope valid
source_connections	tenant, source, credential reference, mode, status	No plaintext secrets in database exports
ingestion_batches	UUID, tenant, source, content hash, mapping version, status	Immutable input and idempotent replay
source_rows	batch ID, row ordinal, source ID, raw	Raw evidence immutable; no cross-tenant

Entity / fact	Keys and critical columns	Required invariants
	payload reference, parse status	joins
fact_sales_lines	tenant, source row, business date, item, signed qty, signed revenue, currency	Duplicate prevention and exact monetary math
fact_orders (optional)	tenant, order ID, opened/fulfilled timestamps, channel, guest surrogate	Do not assume from line-only CSV
dim_products, dim_categories	effective-dated source map, canonical ID, business scope	No silent category reassignment
daily_sales_aggregates	tenant, reporting day, metric version, revenue, units	Reproducible from eligible source rows
reconciliation_runs	tenant/day, snapshot, metric version, SQL hash, results	Deterministic evidence and auditable time
reconciliation_cases	run ID, reason, owner, severity, state	Never mutate source sales through UI
inventory_movements (future)	tenant, SKU, qty, unit, movement type, timestamp	Balance reconstructible from signed movements
vendor_invoice_lines (future)	vendor, SKU, quantity, unit price, effective date, approved status	Traceability before cost-based conclusions
recipes / recipe_lines (future)	dish, version, ingredient, yield, quantity, unit	Versioned and effective-dated costing
labor_shifts (future)	site, role, employee surrogate, scheduled/actual hours, cost	Privacy and source provenance
metric_definitions	metric ID, version, expression, grain, units, allowed filters	One governed interpretation per version
audit_events / outbox_events	actor, action, tenant, event ID, created time	Append-only and deduplicated delivery

8.3 Money, refunds, days and accounting semantics

Financial model rule 1: use exact signed amounts. For the existing CSV sales flow, net_sales = SUM(sales_line_items.revenue), net_units = SUM(sales_line_items.quantity) over eligible business/date records. A refund line of -$20 contributes -$20, not +$20; its signed units contribute similarly. These are net *sales* figures, not bank deposits. Use PostgreSQL NUMERIC, Python Decimal, and a deliberate display/rounding policy at the API boundary. If a future source offers gross, discounts, sales tax, gratuity, fees or refunds as separate columns, normalize their definitions without double-subtracting items already represented in net sales.

Financial model rule 2: revenue recognition and settlement are separate. Payment deposits may be offset by processor fees, reserves, chargebacks, tip routing, delays and cash payments. A difference between net sales and payout does not necessarily mean an error. Separate sales_date, order_opened_at, fulfilled_at, payment_captured_at, settlement_at, imported_at and business_date where those fields genuinely exist. Square describes deferred sales and daily close behavior, showing why a single timestamp cannot cover every report. [R05]

Financial model rule 3: reporting day is a governed business policy. Today's OmniStat CSV ingest uses the parsed date supplied by the source. The future platform should record source date and location-local timestamp separately, with an effective-dated rule such as business_day_start_hour=04:00 in America/Chicago. A transaction at 01:30 on Saturday may belong to Friday's restaurant operating day if explicitly configured; it must not silently change historical reports without a metric version or restatement. Daylight-saving transitions require IANA timezone handling and tests for both the skipped hour and repeated hour.

Financial model rule 4: "missing", "zero", and "unknown" are different states. An absent daily summary is null, not $0. No source records is no_data unless a valid zero-sales close report exists. A no-previous-period baseline yields growth N/A, not infinite growth. Margin and AOV remain unavailable until cost and order-level data exist. This distinction is central to truthful analytics.

Worked reconciliation example (illustrative seeded case)

For 10 November 2026, a source fixture has a +$80.00 sale and a -$10.00 refund. The recomputed signed net sales are **$70.00**. If the daily summary stores **$90.00**, define difference = recomputed − stored = −$20.00; the status is **mismatch**. The evidence proves the arithmetic disagreement, but **does not prove** whether the summary is stale, a record was omitted, or a vendor date rule differs. Each hypothesis must be tested against additional evidence. A second fixture day with

no summary but +$25.00 + $15.00 in lines is **missing summary**, not a $40 overstatement; a matched day with +$120 − $20 + $50 has **$150.00** and zero difference. These are test fixtures, not customer financial results.

8.4 Source adapters and safe ingestion

Define a separate adapter for each supported POS schema. Every adapter implements detect_format, schema_version, validate_columns, parse_row, map_event, normalize_money, normalize_business_day, fingerprint, and explain_errors. Persist a manifest with source name, file hash, upload user, schema version, locale/currency, row count, accepted count, rejected count and reconciliation outcome. Duplicate imports should produce a reviewable warning or idempotent success, not double sales. Ingestion uses a staging area and atomic publish; failed imports do not leave half-updated rollups. Validate untrusted CSV for malicious formulas before export and cap file sizes, rows and processing resources.

8.5 Reconciliation performance and query design

For the common daily investigation workload, the first index candidate is (business_id, business_date, id) on the fact table, possibly covering frequently accessed columns if measurement justifies increased write amplification. Use EXPLAIN (ANALYZE, BUFFERS) against a **fixed seed**, compare the same SQL workload and parameter values before/after, inspect index size, write cost, and P95 latency. Use keyset pagination when source-record pages become very deep; offset pagination remains acceptable for smaller reports. business_id must be the leading predicate. Compare aggregates with decimal arithmetic using one transaction snapshot to avoid race conditions between count and page data.

Partitioning by month or location is not an automatic optimization: it complicates ingestion and indexes and should follow evidence. A cache keyed only by day without tenant and metric version is a security bug. Materialized rollups should have a backfill plan and an explicit invalidation/refresh strategy when a late refund or corrected CSV arrives.

8.6 API and error contracts

Public and UI APIs should use consistent pagination, error envelopes, tracing IDs and data_as_of timestamps. Example read response:

```
{
  "reporting_day": "2026-11-10",
  "metric_version": "net_sales.v1",
  "status": "mismatch",
  "stored": {"net_sales": "90.00"},
  "recomputed": {"net_sales": "70.00", "source_rows": 2},
  "difference_recomputed_minus_stored": "-20.00",
  "cause": {"code": "unexplained_difference", "confidence": "unproven"},
  "evidence": {"next_cursor": null, "items": []},
  "freshness": {"as_of": "2026-11-12T18:00:00Z"}
}
```

The example above is a **proposed future contract**, not an assertion about today's API. If the current frontend expects JSON numbers, change money representation only as a versioned API contract, not as a breaking silent migration. Avoid exposing raw employee IDs, customer details, credentials or unrelated tenant metadata in errors. 400/422 means invalid filter, 401 missing authentication, 403 insufficient role, 404 inaccessible resource (policy-dependent), 409 conflicting mutation, 429 rate limit, 503 dependency unavailable.

8.7 Security, privacy and governance design

Enforce role and tenant checks at each API boundary and in database predicates; use tenant-aware integration tests for cross-business record IDs and upload IDs. Server-generated query plans must not allow SQL injection through arbitrary group names, category filters or order-by clauses. Lock down export access and set presigned URL expiry. JWT secrets are local deployment secrets, never committed. Store audit history of privilege changes, user-visible case annotations and approvals. Use CSRF protection where browser cookies are used, tight CORS origins, log redaction, sensible rate limits and uploaded-content validation. Consider PostgreSQL row-level security as defense in depth once migration and connection-pool semantics are understood. Metabase and Power BI both document tenant/row-level permission patterns; OWASP explicitly calls out object-level authorization risks. [R26, R27, R33]

Privacy-sensitive guest and workforce modules should be opt-in and designed around minimum necessary collection. Market peer benchmarks require consent and cohort thresholds; Toast's own benchmark documentation suppresses insights when cohorts are too small. OmniStat should not reveal individual businesses through sparse market comparisons. [R03]

8.8 Scalability and cost posture

Prefer application profiling, query plans, and constrained per-tenant workloads before adding infrastructure. Near-term capacity targets can be expressed as **design goals** (not measured SLAs): 99.9% availability after monitoring exists; P95 dashboard query under 2 seconds for the agreed dataset; daily investigation first page under 2 seconds; batch ingest completion under 5 minutes for an agreed file size; zero cross-tenant data exposures; zero unresolved monetary rounding discrepancies in fixture tests. Measure on representative hardware and data, then publish actual percentiles rather than adopting the target as fact.

9. Prioritization model: what to build before what

The 129-item catalog is intentionally larger than any reasonable one-year implementation scope. Prioritize each item using a transparent rubric rather than number of features. A practical score is **Priority Index = 0.30 × operator pain + 0.25 × strategic differentiation + 0.20 × source readiness + 0.15 × engineering feasibility + 0.10 × distribution potential − risk penalty**. Each input is rated 1–5 during a team review. The risk penalty captures privacy, correctness, vendor dependence and long-term maintenance. Scores are decision aids, not empirically validated market demand.

Hard gates precede scores. If a feature requires missing data (e.g., margin without costs or AOV without orders), it cannot ship as a verified metric. If the tenant boundary is unproven, it cannot ship to external customers. If an AI explanation cannot link back to permitted evidence, it remains internal research. If an integration violates its terms or the customer lacks rights to its data, it is out of scope irrespective of apparent ROI.

9.1 Recommended product wedges

Wedge A — Trust before AI. Launch the evidence-driven daily discrepancy investigator and consistent metric definitions as OmniStat's most distinctive initial value proposition. These are difficult to fake and highly demonstrable. Integrations and forecasting should attach to this evidence layer rather than bypass it.

Wedge B — Menu insights without fake profit claims. Make category/product exploration exceptional: clean information architecture, comparisons, contribution waterfalls, outlier explanations, exports, and explicit source freshness. Introduce menu-engineering margin quadrants only after verified recipe costs. Lightspeed illustrates the usefulness of item scorecards, while MarginEdge illustrates the complexity of actual cost attribution. [R07, R13]

Wedge C — Owner's morning decision briefing. Curate five important, verified events rather than dumping 100 KPIs: yesterday's net sales vs baseline, source-data confidence, strongest category movement, unresolved discrepancy, and suggested follow-up. Allow drill-through from each sentence to an underlying metric query and source evidence. Tenzo and Tableau Pulse are useful patterns here. [R19, R25]

Wedge D — Modular connected back office. Grow by adding one reliable input domain at a time, starting with invoice/cost or channel/order data based on actual customer willingness to provide it. Each new data domain should unlock a coherent new decision workflow, not merely a decorative chart.

10. Multi-stage engineering roadmap

10.1 Phase boundaries

Phase	Indicative horizon	Target outcome	Dependency gate
0: Trust foundation	Weeks 1–4	Reproducible financial calculations and secure tenant scope	Stable checkout, dataset, migrations, tests
1: Investigation and analytics UX	Months 2–3	Operators can explain most routine reporting differences	Source provenance and consistent metric registry
2: Connected analytics	Months 4–6	Add one POS connector, location/channel data and scheduled reporting	Vendor agreement + source contracts
3: Cost and operations pilot	Months 7–9	Verified recipe/invoice and/or labor decision workflow	Selected pilot partners with actual source data
4: Forecasts and AI pilot	Months 10–12	Out-of-sample validated forecast and source-backed question answering	Enough history and guarded metric query API
5: Platform and scale	Months 13–18+	Multi-unit access, public integrations and optional benchmark product	Proven adoption, security audit, scale evidence

The calendar is illustrative and assumes a focused small team, active product owner, and accessible test customers. It is not a commitment. Integration approval and customer data acquisition often dominate elapsed time. Teams may swap Phase 3 and Phase 4 depending on whether financial cost data or demand histories become available first.

10.2 Detailed build backlog with acceptance criteria

Each entry below is a proposed deliverable suitable for conversion into a GitHub issue or Trello card. An acceptance criterion is evidence of completion, not a description of code being written. **Effort:** S = small coherent change; M = several related modules; L = multi-system/data change; XL = external integration or new sensitive domain requiring extensive review.

Phase 0 — trust, money, testability

ID	Deliverable	Effort	Definition of done / evidence
P0-01	Publish canonical net-sales and refund metric contract	S	Fixture sale/refund totals identical across KPI, category and report APIs
P0-02	Introduce metric definition version in API outputs	M	Every money metric identifies version, units and reporting-day rule
P0-03	Audit all tenant-scoped SQL and exports	M	Negative tests prove tenant B cannot see tenant A IDs in each endpoint
P0-04	Lock source-to-aggregate reconciliation invariants	M	Seeded day status and exact NUMERIC delta reproducible in PostgreSQL
P0-05	Make CSV imports retry-safe and idempotent	M	Uploading the same approved file twice does not double sales
P0-06	Add source row parser audit and rejected-row UI	M	Operator can inspect errors without exposing raw credentials
P0-07	Create reproducible isolated PostgreSQL CI fixtures	M	Schema + data + negative authorization suite run on clean runner
P0-08	Set query-plan and performance baselines	S	Plan and median/P95 measured against recorded dataset and hardware
P0-09	Add CI secret scanning and dependency security policy	S	Pull requests block accidental committed credentials

Phase 1 — evidence-driven workflows and great UI

ID	Deliverable	Effort	Definition of done / evidence
P1-01	Turn daily discrepancy into durable case workflow	M	Case can be assigned, commented on and closed without editing source sales
P1-02	Build row provenance explorer	M	Report row links to original approved upload and parsing trace
P1-03	Add revenue bridge and weekday comparison	M	Refunds and missing baselines remain correctly labeled
P1-04	Implement consistent cross-filtered drill-down	M	Business/category/product screens agree on sum of the same filtered lines
P1-05	Provide saved report views and shareable URLs	M	Filter state is reproducible without trusting browser tenant IDs
P1-06	Add scheduled digest framework	M	Opted-in owner receives evidence-backed digest once per scheduled period
P1-07	Add an alert rule and acknowledgment system	M	Rules deduplicate, scope by tenant and retain source evidence
P1-08	Design meaningful no-data/stale/error states	S	Every chart distinguishes unknown, no data, empty and loading
P1-09	Add WCAG-minded keyboard and screen reader QA	M	Operator can complete investigation without mouse and without chart-only clues

Phase 2 — one reliable connected data domain

ID	Deliverable	Effort	Definition of done / evidence
P2-01	Discover and select one POS API integration	L	Vendor sandbox and scope reviewed; test imports match CSV totals
P2-02	Implement source connection status and secrets handling	M	Credential rotation, expiration and failed sync are visible
P2-03	Normalize order, location, channel and business-day entities	L	Two vendor formats produce comparable, traceable normalized rows
P2-04	Introduce incremental sync and safe backfill	L	Repeated syncs and historical backfills yield identical aggregates
P2-05	Build channel and location dimension explorers	M	Filters preserve tenant and location constraints
P2-06	Implement batch freshness/error alerts	M	Missing expected source feeds raise actionable, low-noise warning
P2-07	Add delivery channel gross-to-net bridge	L	Fee data is source-backed and not conflated with sales
P2-08	Introduce multi-business-day calendar tests	M	Overnight and DST boundary events follow configured policy
P2-09	Deliver configurable weekly management report	M	Snapshot identifies tenant, metric version, watermark and date policy

Phase 3 — inventory, invoice and labor pilots

ID	Deliverable	Effort	Definition of done / evidence
P3-01	Implement canonical unit-of-measure and ingredient catalog	L	Conversion tests cover mass, volume and purchased pack units
P3-02	Add versioned recipe bill of materials	L	Recipe cost histories remain reproducible after edits
P3-03	Add vendor invoice ingestion and review	XL	Human can correct and approve uncertain extracted line fields
P3-04	Build invoice price movement dashboard	M	Price changes trace to vendor invoice evidence
P3-05	Introduce physical inventory movement ledger	L	Balance matches replayed movements across corrections
P3-06	Create theoretical versus actual usage pilot	L	Output distinguishes purchases, usage, waste and transfers
P3-07	Build margin visibility with source confidence badges	L	No margin shown for missing cost inputs
P3-08	Pilot labor timecard import	L	Employee access policy and shift boundary tests pass
P3-09	Add labor cost / net sales dashboard	M	Actual vs estimated labor never mislabeled

Phase 4 — explanation, prediction and safe AI

ID	Deliverable	Effort	Definition of done / evidence
P4-01	Create rolling backtest and baseline forecast harness	M	Evaluation shows results across sites and seasons without leakage
P4-02	Publish demand forecast API with intervals	L	Version, interval coverage, training cutoff and fallback returned
P4-03	Add inventory or labor what-if calculator	M	Assumptions editable, output not represented as causally proven
P4-04	Build governed metric query compiler	L	Only allowlisted tenant-constrained queries can run
P4-05	Introduce read-only natural-language analytics	L	Gold-question accuracy and prompt-injection tests pass
P4-06	Generate source-linked owner morning briefing	M	Every numeric claim opens evidence and shows source freshness
P4-07	Create confirmed-versus-hypothesis reason taxonomy	M	Unknown causes remain unknown in UI/API
P4-08	Add AI monitoring and evaluation dashboards	M	Hallucination, latency and security failure rates tracked

Phase 5 — multi-unit product and extensibility

ID	Deliverable	Effort	Definition of done / evidence
P5-01	Build organization/location hierarchy	L	Cross-location filters and permissions work with mixed roles
P5-02	Add scoped client API keys and versioned REST docs	L	API rate limits, revocation and audit verified
P5-03	Add notification/webhook partner integrations	L	Retries, idempotency and signatures tested
P5-04	Add enterprise sign-in only when demand established	XL	SSO identity mapping and offboarding tested
P5-05	Build mobile-first owner	M	Main flow usable on small

ID	Deliverable	Effort	Definition of done / evidence
	brief and incident view		devices and slow connections
P5-06	Offer opt-in market benchmark with cohort safeguards	**XL**	Minimum cohort thresholds prevent inference of any single business
P5-07	Optimize heavy fact queries using measured plans	**M**	Same dataset/workload compared before and after without write regression
P5-08	Run external security and privacy readiness review	**L**	All critical/high severity findings resolved before enterprise rollout

Backlog size: 52 scoped work items. These are not yet approved sprints. Convert them to issues only after business review, schema inspection and resource estimation. # 11. Metrics and definitions: the contract behind every advanced feature

A single, discoverable **metric registry** is the critical technical investment for trustworthy cross-page analytics. Each entry needs an identifier, purpose, source eligibility, aggregation grain, filter rules, allowed dimensions, unit, data sufficiency requirements, business-day version, null behavior, authorization scope, display precision, owner and change history. The registry is conceptually aligned with Looker's and dbt's documented semantic-layer approach. [R24, R29]

Metric	Definition / denominator	Required source	State when unavailable
Signed net sales	SUM(signed eligible sales-line revenue)	POS sales lines	no_data / stale
Net units	SUM(signed eligible quantities)	POS quantity lines	no_data
Sales growth	(current − prior) / prior × 100	Comparable date windows	N/A when prior ≤ 0; state reason
Revenue share	Category net / sum of positive category net sales	Reliable category mapping	N/A for nonpositive denominator; show signed total separately
Revenue per net unit	Net sales / positive net units	Product quantity and revenue	N/A for zero/nonpositive units; *not* menu price
Gross sales	Pre-discount, pre-refund amount under specified contract	POS gross field	Unavailable from net-only CSV
Refund rate	Signed refund magnitude / appropriate sales base	Classified return events and baseline	Unavailable without refund event type
Order count	Distinct valid completed order IDs	Order-level source	Unavailable from product lines alone
Average order value	Eligible order net sales / completed order count	Unique order IDs	Unavailable without order data
Contribution margin	Net item sales − attributed variable costs	Recipe, prices and allocation rule	Unavailable without costs
Food cost percentage	Approved relevant food cost / comparable food sales	Ingredient/invoice and sales data	Unavailable without cost accounting
Labor cost percentage	Actual eligible labor expense / appropriate net sales	Payroll/timecard and revenue	Unavailable without labor costs
Sales forecast accuracy	Backtested error and interval coverage	Past forecasts + subsequent actuals	Not reported without out-of-sample evaluation
Daily discrepancy	recomputed net − stored daily net	Source lines + stored summary	N/A if summary is missing
Data completeness	Accepted expected source coverage / declared expected input	Source contract and manifest	Unknown if expectation undefined
Guest repeat rate	Identified returning guest count / identifiable guests	Consented stable guest keys	Unavailable from anonymous sales lines

Attribution contract. Gross sales, net sales, settlement, revenue and profit are not interchangeable. Every revenue visual should identify its scope (business, location, period, currency, source systems), period completeness, and whether returns are signed. A revenue change is not itself a causal explanation; drivers may decompose an arithmetic change without proving underlying behavior. This is especially important for generative AI summaries.

12. Competitive differentiators: where OmniStat can win

12.1 Positioning versus bundled POS suites

A POS suite's strength is control over the transaction and payment flow. OmniStat's potential strength is **cross-vendor neutrality**, faster onboarding for businesses that already have a POS, and explanations spanning multiple disconnected inputs. Selling an all-purpose POS replacement would require payment regulation, hardware servicing, offline transaction processing, detailed kitchen workflows, liability and intensive support. That is a different company. A vendor-neutral "explain and improve the business" layer is more achievable in the near term.

Positioning statement (hypothesis): *OmniStat helps independent and growing restaurants turn fragmented POS exports into verified daily revenue, explainable product insights, and action-oriented operating decisions—without changing their POS.* This should be tested with customers before being declared proven.

12.2 Differentiation by evidence, not just model sophistication

Competitive category/product dashboards are now common. Stronger differentiation comes from a deep source-to-KPI chain: every surprising number includes **what was calculated, from which data, under which rules, at what time, and with which gaps**. The product should expose a metric "receipt," backed by a versioned query and source lineage. That becomes the foundation for trustworthy AI and potentially for finance audit collaboration.

12.3 Avoid the feature-count trap

A full kitchen-display system, on-premise POS, staffing SaaS, loyalty network, payments integration, invoice OCR, and enterprise warehouse could require years of support commitment. The research catalog is a library of opportunities. It should not become an indiscriminate sprint contract. Favor workflows that demonstrably resolve costly decisions and build reusable infrastructure for the next workflow. If a proposed capability cannot be explained in terms of input, user decision, validation and measurable outcome, it is not ready.

12.4 Competitor signals worth continuously monitoring

Watch for: vendor changes in data export scope and API access; new AI action permissions and beta/GA status; price and plan packaging; restaurant-specific day-close and payment-reconciliation features; changes in forecasting explainability; interoperability and vendor partnerships; data privacy terms; and availability of real-time/near-real-time reports. Public documentation provides directional signals, not proof of adoption. Update this paper quarterly with a dated change log and a new source citation for material changes.

13. Product validation and go-to-market research agenda

13.1 Whom to interview

Recruit a small, intentionally diverse panel rather than assuming a single restaurant workflow: independent counter service, full-service owner-operator, multi-unit manager, café/bakery, bar/beverage program, and finance/bookkeeper user. Include at least one operator whose current reporting already works well; understanding what they would **not** switch for is useful. Target 12–20 discovery interviews initially, followed by 3–5 pilot sites with sample data. These are research planning targets, not completed interviews.

Suggested interview protocol: Ask the owner to show their last daily close; where totals disagree; how they handle refunds, voids, late sales and order channels; which source is trusted when numbers differ; the time spent on reconciliation; how they decide menu and labor changes; whether ingredient costs exist by item; acceptable integration steps; data security concerns; which 2–3 alerts would actually trigger action; and current tools paid for. Do not begin by asking whether they "want AI." Ask what decision needs a better answer.

13.2 Prototype and usability study

Test the flow Dashboard → date selection → stored/recomputed totals → source evidence → case note. Have the user independently answer: Is the report matching? What is the signed difference? Was there a refund? Which batch contributed? Is the root cause proven or merely suspected? Time-to-correct-answer, number of navigation

errors and confidence calibration are more informative than visual preference alone. Repeat for category → product drill-down with filter preservation and a no-data scenario.

13.3 Pilot success metrics

Goal	Measurement	Failure/guardrail
Reduce revenue investigation burden	Median minutes from alert to understood evidence	No fabricated “resolved” causes
Improve reporting trust	% of audited days with reproducible source-to-summary agreement	No tenant leakage or hidden line items
Produce useful owner insights	% of digest recipients who investigate or act on a high-value insight	Alert opt-out / overload
Shorten onboarding	Minutes from first valid CSV to a verified dashboard	Mapping errors and misleading auto-guesses
Improve operational reporting	Time saved preparing a weekly management report	Maintain owner confidence and line traceability
Validate forecasts	WAPE/MASE and interval coverage versus naive baselines	No evidence of model advantage → do not market predictive superiority
Validate secure access	Cross-tenant negative tests and access audit	Any data isolation failure blocks release
Validate economics	Measured support load, cloud cost and willingness to pay	Features without cost recovery are deferred

13.4 Packaging hypothesis

Offer a clear low-friction **Core** tier with CSV upload, business KPIs, category/product drill-down and data-quality status; a **Professional** tier with verified multi-source ingestion, case management, scheduled reporting, custom views and alerts; and a **Connected Operations** tier with permitted POS, invoice/labor integrations, margin workflows and forecasting. An enterprise package may later add multi-location hierarchies, SSO, APIs, privacy controls and benchmarking. **No price points are proposed here** because price sensitivity and willingness to pay require customer evidence. Vendor list prices should not be treated as directly comparable—payment processing, hardware, usage and add-ons differ.

14. Quality engineering: proof before release

14.1 Unit and contract testing

At minimum, write golden tests for signed refunds, duplicate lines, invalid dates, zero and missing summaries, cross-business isolation, missing category, negative-only categories, decimal precision, CSV mapping variants, leap day, DST, duplicate uploads, late events, and API error envelopes. Test a metric once at the semantic layer and again across dashboard/API outputs for the same seed. Any cost/margin calculation requires explicit units and effective-dated cost history. Any AI text that quotes a number should be traceable to the exact metric response.

14.2 Real PostgreSQL integration testing

Use a disposable database with tracked schema migrations and deterministic fixtures. Run import → raw row → normalized line → summary → reconciliation → paginated source evidence end to end. Negative test another tenant’s business, source row, upload ID, export and cached metric query. For critical financial comparisons, use a consistent transaction snapshot.

Capture query plans with `EXPLAIN (ANALYZE, BUFFERS)` and verify indexes on fixed data sizes. Include rollback tests, failed import recovery, and a fresh-install smoke test rather than relying solely on developer machines.

14.3 Performance and reliability testing

Define three representative workloads: dashboard for 30 days; category/product detail for a high-volume business; and deep paginated daily investigation. Record row counts, index definitions, PostgreSQL version, hardware, warm/cold run conditions, SQL parameters and five or more timing measurements. Compare before and after tuning **the same dataset and queries**. Load tests need multiple concurrent tenants and must verify that cache or pooling does not cross-leak data. Introduce p95 and p99 indicators, error rates, slow query alerts, queue lag and ingestion data freshness dashboards. Publish only measured numbers.

14.4 Frontend and accessibility testing

Use realistic data states: one correct day, one mismatch, one missing summary, a refund-only day, an empty business, a high-cardinality product category, and a source with missing cost fields. Test navigation, keyboard operation, screen-reader labels, mobile layout, date parsing, loading/error states and pagination under slow responses. Browser checks should confirm that selected 7/30/90-day windows and displayed currency align with API contracts. A screenshot is evidence of rendering, **not** evidence of correct financial calculations.

14.5 AI and forecast validation

Partition training and test sets by time; where possible, hold out unseen businesses or locations. Compare with simple baselines, evaluate low-volume and high-volume sites separately, inspect calibration, and avoid data leakage from future corrections. For AI answers, build an auditable bank of questions with expected permitted results and explicit *cannot answer* cases. Red-team malicious content in product names and CSV cells, cross-tenant requests and attempts to trigger unauthorised operations. AI-generated SQL must be constrained to the metric compiler and read-only database role. Shadow evaluation should precede customer-visible publication.

14.6 Release evidence checklist

A feature is release-ready only when: schema change is reviewed and reversible where feasible; migration applies on fresh and upgraded database; required metrics are defined; tenant isolation tests pass; source and money semantics are consistent; realistic PostgreSQL tests pass; frontend is usable on mobile; performance measurements are recorded; sensitive credentials remain outside Git; change notes explain new behavior; and an owner can demonstrate the outcome against a known fixture. The implementation and benchmark recently developed for daily reconciliation is a useful template for this level of evidence.

15. Risk register and engineering tradeoffs

Risk	Why it matters	Proposed mitigation / review trigger
Metrics drift across dashboards	Same business receives contradictory net sales	Single semantic definition and golden cross-screen fixtures
Missing inputs masquerading as zero	Fake margins, rates or growth	Explicit unavailable/unknown states and source coverage checks
CSV duplicates and late refunds	Overstatement or restatement of revenue	Idempotency, source snapshot and dated version rules
Tenant data exposure	Serious customer/security harm	Owner-bound SQL, RLS when appropriate, adversarial tests, audit
AI hallucinations or prompt injection	False financial conclusions or dangerous actions	Governed query tools, citations, approvals, red-team evaluations
Forecast overfitting / insufficient history	Unreliable staffing or ordering advice	Baselines, confidence intervals, minimum sample gates
Vendor API dependence	Sudden schema/pricing/access change	Adapter versioning, export fallback, integration health alerts
OCR invoice misread	Wrong cost / vendor approval	Human review on low-confidence values, source image trace
Privacy rights / retention	Regulatory and trust risk	Data minimization, consent records, deletion and export policy
High support burden	An extensive feature surface becomes unmaintainable	Narrow initial wedge, measurable feature adoption
Slow deep pagination	Owner cannot review large daily evidence	Query plans, covering indexes, keyset when justified
Complexity from premature microservices	More failure modes than customer value	Modular monolith until operating thresholds justify split
Misleading market benchmarks	Competitor data leakage or bad comparison	Opt-in cohorts, minimum sample, suppression, differential privacy review

Representative tradeoff A — build vs buy charts. A library such as Chart.js may be sufficient for existing dashboards; adopting a full embedded BI vendor introduces licensing, SSO, tenancy and operational constraints. Start with reusable OmniStat chart/metric components, benchmark requirements and use cases; evaluate commercial embedding only if customer self-service demand outgrows in-house capacity.

Representative tradeoff B — custom POS adapters vs ETL service. Handcrafted connectors offer tight domain semantics but high vendor-maintenance cost. A general integration framework increases coverage but may lose business-specific refund/date fields. Use a normalized adapter contract and choose per-connector based on access rights, quality and expected customers; do not assume a broad connector count guarantees useful POS semantics.

Representative tradeoff C — exact finance vs speed. Financial reconciliations prioritize correctness and evidence over sub-100-ms display, while high-traffic trend charts can use versioned rollups. A dashboard cache should never be the sole source of truth for an audit. Use measured workloads to choose materialization and caching, not optimism.

16. Proposed 90-day leadership agenda

Days 1–15: Audit current code and production schema; establish the signed-revenue contract; create one clean tenant-isolation matrix; verify the daily discrepancy branch and actual seed/benchmark; interview at least three owners about daily close. Produce a documented backlog and delete any duplicate route or legacy calculation. No AI features yet.

Days 16–30: Ship a reliable source evidence explorer, meaningful mismatch reason codes, and clean report UX. Add ingestion idempotency and simple stale/missing file flags. Confirm each UI number matches PostgreSQL and the metric registry. Prepare a single documented pilot dataset and browser demo.

Days 31–60: Pilot with three distinct restaurants; instrument time-to-investigate and reporting errors; add saved views and one concise scheduled digest. Select one POS export/API integration only after mapping source permissions and business-day semantics. Add alerts with owner acknowledgment and deduplication.

Days 61–90: Release the first integrated vendor source if permitted, run customer usability and security reviews, and measure production query performance. Decide—using real customer interest—whether to pursue recipe/invoice economics or shift-level demand next. Produce a defensible, source-backed case study with quantitative outcomes only after they have been measured.

17. Research gaps and future studies

This research does not establish total addressable market, willingness to pay, customer churn, exact competing feature adoption, commercial feature entitlements, or revenue impact of OmniStat. Those require interviews, product trials, vendor quotes and carefully designed experiments. Another gap is the cross-source interpretation of refunds and deferred orders: even when two POS systems export a column named “net sales,” the actual calculation may differ. Perform source-field contract tests instead of assuming semantic equivalence by column name. A final gap is operational economics: invoice OCR, labor integration, mobile notifications and LLM inference all add recurring cost and support burden. Develop a unit-economics model before committing to unlimited usage promises.

Future research questions: (1) Which recurring financial discrepancy consumes the most operator hours? (2) How often are reporting issues caused by missing/late files versus calculation errors? (3) Which combination of metrics genuinely changes menu or staffing decisions? (4) What is the minimum viable history for useful restaurant forecasts? (5) Can explanation-first UX improve operator confidence without overstating causality? (6) How much vendor-neutral data connection friction will users tolerate? (7) Does a source-backed AI briefing outperform a well-designed deterministic report on accuracy, speed and trust?

18. Final recommendations

Build the next generation of OmniStat around **proof, depth, and action**. Keep the strong existing sales intelligence and category-to-product navigation. Finish the daily discrepancy investigation as an auditable case workflow, because it is a concrete differentiator and technical demonstration of data integrity. Make ingestion stable and metric definitions reusable. Add deep channel/location views only when correct source fields exist. Choose inventory/recipe costs or labor planning based on actual pilot partners. Introduce forecasts only after out-of-sample validation. Introduce conversational AI only when it can query governed metrics and show evidence. Add human-approved workflows after alerts prove useful.

The right ambition is not to say OmniStat has more features than Toast, Square, Restaurant365, Tenzo or Looker. It is to deliver **a dependable financial and operational intelligence experience across the tools restaurants already use**, with unusually clear answers to: *What happened? Which records prove it? What is missing? What is likely next? What action should a responsible operator consider?*

Appendix A. Evidence coding and reproducibility

Every [Rxx] points to a publicly accessible official product or technical document listed below. The code is a **source locator**, not a quantified score of feature quality. Company marketing pages should be understood as published claims; support documentation often offers better implementation detail but can be versioned or subscription-specific. Recheck sources when planning a particular vendor integration. Proposed features and design decisions are labeled **I** or described as hypotheses. Known OmniStat behavior is **V**—observed from prior shared source snapshots and development reports, not guaranteed to match the latest uninspected deployment.

For reproducible internal planning, maintain an evidence_registry record with source_url, vendor, feature_claim, retrieved_on, plan_or_region, release_state, researcher, and verification_note. The paper's next edition should cite any operator interviews and real product demonstrations separately from official marketing documents. Always retain exact benchmark commands, hardware/environment, dataset and observed results alongside claims.

Appendix B. Operator research interview questions

1. Show the last POS daily close and the report you believe most. When did it last disagree?
2. How are refunds, voids, discounts, sales tax, tips and payment fees represented in your existing tools?
3. What is the actual "business day" when your restaurant closes after midnight?
4. Which reports are manually combined every week? Which column-level fields are missing?
5. What action did you take last time one product category declined? What evidence supported it?
6. Would you upload invoice files and recipes to receive food-cost insights? Who approves the numbers?
7. How do you schedule staff? Would a forecast change the schedule, and who is accountable?
8. Do you have consented repeat-guest identifiers, and do you want to use them for analytics?
9. Which daily alerts would you welcome? What is already too noisy?
10. Would you permit read-only connections to your POS/ERP? Which permissions feel excessive?
11. What does a successful first five minutes after signup look like?
12. What would make an AI explanation trustworthy, and when would you never act on it?
13. What does your business currently pay for reporting, inventory and scheduling tools?
14. What data must never be used for market benchmarks or model training?
15. Who besides the owner should be permitted to view financials, refunds, margins and labor details?

Appendix C. Proposed architecture decision records (ADRs)

ADR-001 — Modular monolith. Decision: keep FastAPI + PostgreSQL modules and a background job service until measured load supports a split. Alternatives: per-domain microservices, immediate full warehouse. Tradeoff: slightly tighter deployment coupling in exchange for operational simplicity and consistent transactions.

ADR-002 — Metrics defined centrally. Decision: metric registry with versioned formulas and business-day policy. Alternatives: hard-coded calculation per chart. Tradeoff: upfront modeling cost in exchange for coherence, API stability and trustworthy AI.

ADR-003 — Evidence precedes recommendations. Decision: every reconciliation status links to source rows; cause is unproven absent deterministic evidence. Alternatives: model-generated narrative without provenance. Tradeoff: fewer flashy answers, substantially better auditability.

ADR-004 — Tenant-scoped queries. Decision: derive tenant from authorized identity, bind in SQL, explore PostgreSQL RLS as defense in depth. Alternatives: filter only in the browser. Tradeoff: more tests and connection management, but necessary isolation.

ADR-005 — Transaction-level integrity in ingestion. Decision: imports are idempotent, raw evidence retained under policy, and publish uses a transaction. Alternatives: best-effort row inserts. Tradeoff: additional staging/manifest engineering versus reliable finance numbers.

ADR-006 — Defer true profit and churn claims. Decision: leave unavailable until cost and guest/order fields are proven. Alternatives: heuristic guesses. Tradeoff: slower breadth, higher credibility and reduced risk of misleading restaurant operators.

ADR-007 — Read-only AI first. Decision: safe metric-query gateway and evidence-backed response generation. Alternatives: LLM with unrestricted SQL or operational write tools. Tradeoff: limits autonomy until permissions, audit and evaluation are mature.

ADR-008 — Do not optimize blind. Decision: use representative EXPLAIN ANALYZE, application traces and before/after workload measurements. Alternatives: add indexes/partitions speculatively. Tradeoff: greater benchmark discipline in exchange for fewer regressions.

Bibliography: official sources reviewed

Reference URLs were checked during preparation on 12 November 2026. Feature availability and pricing are subject to change; current product terms should be verified before purchase or integration.

[R01] Toast. Restaurant Reporting & Analytics. https://pos.toasttab.com/products/reporting

[R02] Toast. Toast IQ and restaurant AI trends (June 2026). https://pos.toasttab.com/news/toast-iq-restaurant-trends-report-ai-assistant

[R03] Toast. Restaurant Performance Benchmarking. https://pos.toasttab.com/products/benchmarking

[R04] Toast. xtraCHEF invoice and cost management. https://pos.toasttab.com/products/xtrachef

[R05] Square. Square for Restaurants: features, reporting and reconciliation. https://squareup.com/us/en/point-of-sale/restaurants/pricing

[R06] Lightspeed. Advanced Insights 2.0 overview. https://k-series-support.lightspeedhq.com/hc/en-us/articles/18812821372443-About-Advanced-Insights-2-0

[R07] Lightspeed. Advanced Insights: Menu reports and item scorecards. https://k-series-support.lightspeedhq.com/hc/en-us/articles/7337113789979-About-Menu-reports

[R08] TouchBistro. Restaurant Reporting & Analytics. https://www.touchbistro.com/features/reporting-analytics/

[R09] Restaurant365. Restaurant accounting, payroll, AP and GL. https://www.restaurant365.com/accounting/

[R10] Restaurant365. AI Dashboards product page. https://www.restaurant365.com/inventory/ai-dashboards/

[R11] Restaurant365. Operations Dashboard documentation. https://docs.restaurant365.com/docs/operations-dashboard

[R12] Restaurant365. Forecasting overview. https://docs.restaurant365.com/docs/forecasting

[R13] MarginEdge. How it works: invoices, recipe, inventory and reports. https://www.marginedge.com/how-it-works

[R14] MarginEdge. Automated invoice processing. https://www.marginedge.com/automated-invoice

[R15] MarginEdge. Plate cost alerts. https://www.marginedge.com/lp/plate-cost-alerts

[R16] MarketMan. Restaurant management platform. https://www.marketman.com/platform

[R17] MarketMan. Pricing and plan-feature comparison. https://www.marketman.com/pricing-for-restaurant-inventory-management-system

[R18] 7shifts. What is 7shifts? (knowledge base). https://kb.7shifts.com/hc/en-us/articles/53059862558099-What-is-7shifts

[R19] Tenzo. Restaurant reporting and intelligence. https://www.gotenzo.com/

[R20] WISK. Feature and plan comparison. https://www.wisk.ai/new-pricing

[R21] Craftable. Restaurant purchasing, inventory and vendors. https://craftable.com/restaurants

[R22] Deliverect. Analytics dashboard documentation. https://help.deliverect.com/en/articles/7978951-introduction-to-your-dashboard

[R23] Olo. Guest data platform. https://www.olo.com/gdp

[R24] Google Cloud Looker. Embedded and conversational analytics, semantic layer. https://cloud.google.com/looker

[R25] Tableau. Tableau Pulse personalized metric intelligence. https://www.tableau.com/products/tableau-pulse

[R26] Metabase. Embedded interactive dashboard and tenant controls. https://www.metabase.com/docs/latest/embedding/dashboard

[R27] Microsoft Power BI. Row-level security with Power BI. https://learn.microsoft.com/power-bi/service-admin-rls

[R28] Microsoft Power BI. Incremental refresh overview. https://learn.microsoft.com/en-us/power-bi/connect-data/incremental-refresh-overview

[R29] dbt Labs. Semantic Layer and metric definitions. https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl

[R30] dbt Labs. Data tests for model integrity. https://docs.getdbt.com/docs/build/data-tests

[R31] dbt Labs. Source freshness checks. https://docs.getdbt.com/docs/deploy/source-freshness

[R32] Airbyte. Data integration capabilities and CDC. https://airbyte.com/product/capabilities

[R33] OWASP. API Security Top Ten: object-level access and other risks. https://api-security.owasp.org/editions/2023/en/0x11-t10/

[R34] Toast. Q1 2026 restaurant AI/POS trends report. https://pos.toasttab.com/blog/data/q1-2026-restaurant-ai-pos-trends

[R35] Restaurant365. Forecast Analysis documentation. https://docs.restaurant365.com/docs/forecast-analysis

[R36] Square. Restaurant feature matrix with Square AI beta. https://squareup.com/us/en/point-of-sale/restaurants/pricing

[R37] Craftable. Analytics documentation and report library. https://help.craftable.com/learning/analytics

[R38] Toast. Full platform overview. https://pos.toasttab.com/

[R39] Deliverect. Developer API and integration hub. https://developers.deliverect.com/